

DeepSeek-R1 这篇论文的核心贡献是：它证明了一个足够强的 base model，不一定必须先靠大量人工 CoT 标注做 SFT，也可以通过“可验证任务 + 规则奖励 + 大规模强化学习”，自发学出长链推理、反思、验证、换方法等 reasoning 行为。

论文里最关键的实验对象是 **DeepSeek-R1-Zero**：它跳过传统 SFT，直接在 DeepSeek-V3-Base 上做 RL，最后在数学、代码、STEM 等可验证任务上出现了明显的推理能力提升。论文明确说，它们没有显式教模型怎么推理，但模型通过 RL 自然发展出 verification、reflection、alternative approaches 等行为。

1. DeepSeek-R1-Zero 是什么？

DeepSeek-R1-Zero 是一个“几乎不靠人工推理示范、直接用强化学习激发 reasoning 能力”的模型。

更准确地说：

它的起点是 **DeepSeek-V3-Base**，也就是一个已经预训练好的 base model。传统路线通常是：

Base Model → SFT → RLHF / RL → Chat Model / Reasoning Model

而 DeepSeek-R1-Zero 的路线是：

DeepSeek-V3-Base → 直接 RL → DeepSeek-R1-Zero

论文特别强调：R1-Zero 绕过了 **conventional supervised fine-tuning**，也就是传统 SFT 阶段。这样做的原因是，作者认为人类写出来的推理路径可能会限制模型探索更有效、甚至非人类式的推理策略。

它训练时主要用了两类奖励：

- 准确性奖励**：最终答案对不对。
例如数学题答案能否匹配标准答案；代码题能否通过测试用例。
- 格式奖励**：模型是否按要求把推理过程放在 `<think>...</think>`，把最终答案放在 `<answer>...</answer>`。

也就是说，它不是被教：

“你应该先列条件，再推公式，再检查答案。”

而是被奖励：

“你最终答对了，而且格式符合要求。”

通俗理解：

R1-Zero 像一个学生，不是先看老师写的标准解题步骤，而是不断刷题。==只要最后答案对，就得分；答案错，就不得分。刷得足够多之后，它自己摸索出了：先多想一会儿、检查一下、发现错误就重算、换一种方法试试。==

这就是 R1-Zero 最有意思的地方：它把 reasoning 从“模仿人类推理文本”推进到了“通过结果反馈自我进化推理策略”。

2. 为什么 pure RL 能激发 reasoning?

这里的 **pure RL** 指的是：不先做 reasoning SFT，不先给大量人工 CoT 示例，而是直接通过强化学习来优化模型。

它能激发 reasoning，核心原因有三个。

2.1 推理任务有“可验证答案”

数学、代码、逻辑、部分 STEM 题都有一个特点：

中间怎么想不一定容易判断，但最终答案对不对比较容易判断。

例如：

数学题：答案是不是 42？

代码题：能不能通过测试用例？

选择题：选项是不是 B？

这类任务天然适合 RL，因为奖励信号清晰。

通俗说：

写作文很难自动打分，但做数学题、跑代码测试很容易知道对错。只要能知道对错，模型就可以通过大量试错来学习。

论文也明确说，DeepSeek-R1-Zero 的 rule-based reward 主要用于数学、代码、逻辑推理等领域，其中数学用答案匹配，代码用编译器和测试用例来验证正确性。

2.2 模型本身已经有“潜在推理能力”

R1-Zero 不是从零训练出来的，它基于 DeepSeek-V3-Base。这个 base model 在预训练阶段已经见过大量数学、代码、英文、中文、技术文本，也具备生成合理解题过程的基础能力。

所以 RL 做的不是“凭空创造 reasoning”，而是：

从 base model 已经可能生成的众多行为中，把更容易答对题的行为强化出来。

通俗说：

base model 已经会一点解题，只是不稳定。RL 像一个筛选和强化机制：凡是更容易得到正确答案的推理方式，就让它以后更常出现。

2.3 长 CoT 是一种有效的“测试时计算”

在 RL 过程中，R1-Zero 的平均输出长度不断变长。论文图 1 显示，随着训练推进，模型回答长度明显增加，AIME 2024 的 pass@1 从 15.6% 提升到 77.9%；用 self-consistency 还能到 86.7%。论文解释说，模型自然学会用更多 thinking time 来解决 reasoning 任务。

这意味着模型学到了一个策略：

难题不要马上答，先多推几步、检查几次、尝试不同路径。

通俗说：

以前模型像考试时看到题就抢答；RL 之后，它发现“多打草稿、多验算、多换方法”更容易拿分，于是它开始主动延长思考过程。

这就是 pure RL 激发 reasoning 的本质：

不是奖励“看起来像推理”，而是奖励“推理之后真的更容易答对”。

3. “aha moment” 是什么意思？

论文里的 **aha moment**，可以理解为：

模型在 RL 训练中突然表现出一种“自我反思 / 发现错误 / 重新审题”的行为模式。

论文第 5 页表 2 给了一个例子。模型在解一道数学题时，推到一半突然写出类似：

“Wait, wait. Wait. That’s an aha moment... Let’s reevaluate this step-by-step...”

它意识到前面的推导可能有问题，于是停下来重新检查。论文还提到，在训练过程中，模型使用“wait”这类反思性词语的频率突然增加，这被作者视为 R1-Zero 自我进化过程中的一个标志性变化。

这个“aha moment”不是说模型真的像人一样有意识，而是说：

从行为上看，它学会了在推理过程中发现可疑步骤，并主动修正。

通俗解释：

假设一个学生解题时一开始这样写：

“先把两边平方……然后展开……”

写着写着突然发现：

“等等，我刚才平方后可能引入了增根，我得回头检查一下。”

这就是人类解题中的 aha moment。

对模型来说，aha moment 表明它不只是机械地生成步骤，而是出现了类似：

1. 我先尝试一种方法；
2. 我发现这个方法可能错；
3. 我停下来检查；
4. 我换一种方式继续。

这类能力非常重要，因为真正的复杂推理往往不是一条直线，而是：

尝试 → 犯错 → 发现问题 → 修正 → 再验证。

R1-Zero 的价值就在于：这些行为不是人工写进数据里教出来的，而是在 RL 奖励下自然出现的。

4. GRPO 大概解决什么问题？

GRPO，全称 **Group Relative Policy Optimization**，是 DeepSeek-R1 / R1-Zero 使用的强化学习算法。

你可以先把它理解成：

GRPO 是一种比 PPO 更省资源、更适合训练长 CoT reasoning model 的 RL 算法。

传统 RLHF 常用 PPO。PPO 里通常有两个重要模型：

1. **Policy Model**：正在训练的模型，负责生成答案。
2. **Value Model / Critic Model**：评估当前生成到一半时，未来大概能拿多少 reward。

问题是，在大模型 reasoning 场景下，Value Model 很麻烦：

- 它通常和 policy model 差不多大，训练它很耗显存和算力；

- 长 CoT 里，前面写的内容可能后面被推翻，所以很难根据“生成到一半的文本”判断最终 reward；
- 只有最终答案对错时，中间每一步该给多少价值也很难估。

论文在 Appendix A.3 明确说，GRPO 放弃了 value model，而是用一组输出的 reward 来估计 advantage，从而降低 PPO 的资源消耗。

GRPO 的核心做法是：

对同一道题，让模型一次生成多个答案，比如 16 个；然后看这一组答案里，哪个比平均水平好，哪个比平均水平差。

如果某个答案 reward 高于组内平均，就提高它的概率；
如果低于组内平均，就降低它的概率。

通俗解释：

PPO 像是给每个学生配一个“预测老师”，这个老师要在学生解到一半时判断：你最后大概率能考几分。这个老师很贵，也很难训练。

GRPO 则更简单：

同一道题让 16 个学生各做一遍。最后谁答对了、谁答错了，一比较就知道谁更好。不需要额外训练一个预测老师。

所以 GRPO 解决的核心问题是：

1. **降低训练成本**：不需要额外 value model。
2. **适合 outcome reward**：最终答案对错就可以形成组内比较。
3. **适合长 CoT**：不用强行判断中间每一步的价值。
4. **提升大规模 RL 可行性**：让 reasoning model 的 RL 训练更工程可落地。

这也是 DeepSeek-R1 论文的关键工程贡献之一：它不是只提出“RL 能激发推理”，还给了一个更适合大规模训练的 RL 算法框架。

5. R1 和传统 SFT 模型的差异

这里要区分两个对象：

- **传统 SFT 模型**：主要靠人工或高质量样本示范来学会回答。
- **DeepSeek-R1 / R1-Zero**：更多通过 RL，让模型围绕正确性、可验证性和偏好奖励自我优化。

5.1 传统 SFT：学的是“模仿标准答案”

SFT 的本质是：

给模型很多「问题 → 标准回答」样本，让模型学会模仿这些回答。

它的优点是稳定、可控、可读性好。比如你希望模型礼貌、结构清晰、遵循指令，SFT 很有效。

但它的问题是：

模型学到的是人类提供的答案分布。如果人类答案没有写出充分反思、验证、试错过程，模型也很难学到这些能力。

论文在 Appendix A.2 里说，SFT 可能阻碍模型探索和发展有效 reasoning 策略，因为人类提供的回答不总是最适合模型学习，且常常省略显式反思和验证步骤。

通俗说：

SFT 像临摹字帖。字帖写得好，学生就写得像；但如果字帖没有展示草稿、验算、反思，学生也不一定会学会真正的解题策略。

5.2 R1-Zero：学的是“怎样更容易答对”

R1-Zero 不先模仿人工 CoT，而是通过 RL 得到反馈：

哪些生成方式更容易让最终答案正确？

所以它学到的不是固定模板，而是一类策略：

- 多想；
- 检查；
- 反思；
- 换方法；
- 验证答案。

论文说，R1-Zero 会生成更长回答，并在回答中加入 verification、reflection、alternative approaches。

通俗说：

SFT 是“看老师怎么写”；R1-Zero 是“自己刷题，靠得分信号总结出怎么写更容易对”。

5.3 R1：在 R1-Zero 基础上补齐可读性和通用能力

不过 R1-Zero 也有明显缺点：

- 可读性差；
- 中英文混杂；
- 主要擅长数学、代码、逻辑等 reasoning 任务；
- 写作、开放问答、通用对话能力不足。

所以 DeepSeek-R1 不是纯粹的 R1-Zero，而是一个多阶段训练结果。论文说，R1 通过 cold-start data、RL、rejection sampling、SFT、第二阶段 RL 等流程，让模型继承 R1-Zero 的 reasoning 能力，同时用非 reasoning 数据对齐人类偏好，改善写作、开放问答、helpfulness 和 harmlessness。

所以可以这样理解：

模型类型	核心训练逻辑	优点	缺点
传统 SFT 模型	模仿人工高质量回答	稳定、可控、可读性好	可能受限于人工示范，不一定激发深度 reasoning
R1-Zero	直接 RL，奖励最终正确性和格式	能自发发展长 CoT、反思、验证	可读性差，语言混杂，通用能力弱
R1	R1-Zero 思路 + cold-start + SFT + RL + 偏好对齐	reasoning 强，同时更符合人类使用习惯	训练流程更复杂，仍有 token efficiency、工具使用、语言混杂等限制

6. 蒸馏小模型有什么产业意义？

这部分非常重要，因为它决定了 reasoning model 能不能从“少数大厂的超大模型能力”变成“产业可部署能力”。

DeepSeek-R1 论文专门做了 distillation：用 DeepSeek-R1 生成的数据去微调 Qwen、LLaMA 等较小的开源模型。论文提到，它们使用了 **80 万条由 DeepSeek-R1 生成的样本**，对多个开源 base model 做 SFT 蒸馏，并且蒸馏模型在 reasoning benchmark 上表现很强。

6.1 降低部署成本

大模型训练和推理都很贵，尤其是长 CoT reasoning model。每次回答都可能生成上万 token，推理成本很高。

蒸馏的意义是：

把大模型的 reasoning 行为压缩到小模型里，让企业能用更低算力部署类似能力。

例如，论文里的 DeepSeek-R1-Distill-Qwen-1.5B、7B、14B、32B，以及 Llama-8B、70B 等模型，覆盖了从轻量部署到中大型部署的不同场景。

通俗说：

大模型像顶级专家教授，但每次请他都很贵；蒸馏小模型像把教授的解题方法整理成教材，让普通学生也能学到一部分能力。

6.2 让 reasoning 能力进入本地化和私有化场景

很多企业不一定能接受把数据发给超大云端模型，尤其是金融、医疗、政企、工业、法律等领域。

小模型蒸馏后，可以更容易：

- 私有化部署；
- 本地部署；
- 边缘部署；
- 在企业内部知识库和业务系统中集成；
- 控制数据安全和成本。

这对产业落地很关键。因为很多企业真正关心的不是 benchmark 第一，而是：

能不能在我的服务器上跑？

成本能不能接受？

延迟能不能接受？

数据能不能不出内网？

蒸馏模型把 reasoning model 从“炫技能力”变成了“可部署能力”。

6.3 小模型直接 RL 很难，蒸馏更经济

论文 Appendix F 有一个重要结论：对较小模型来说，直接做大规模 RL 未必比蒸馏更好。实验中，DeepSeek-R1-Distill-Qwen-32B 在多个 reasoning benchmark 上显著超过直接 RL 得到的 Qwen2.5-32B-Zero。论文据此认为：把强 teacher model 的能力蒸馏到小模型上，效果优秀；而小模型自己做大规模 RL 需要巨大算力，还可能达不到蒸馏效果。

这点产业意义很大：

不是每家公司都需要从头训练 R1。很多公司更现实的路线是：用强模型生成高质量 reasoning 数据，再蒸馏到自己的小模型或垂直模型里。

这就形成了一条更可行的产业路径：

6.4 推动开源生态和应用创新

论文也强调，蒸馏模型可以帮助研究社区理解 long CoT reasoning model 的机制，并推动更强 reasoning model 的发展。

从产业角度看，这意味着：

- 创业公司不必从零训练大模型；
- 垂直行业可以基于小模型做定制；
- Agent、代码助手、教育解题、金融分析、数据分析等场景可以更低成本接入 reasoning 能力；
- 端侧 AI 和本地 AI 有更大可能性。

尤其是对 Agent 应用来说，小模型蒸馏很有价值。因为 Agent 往往不是回答一次，而是多轮调用、工具调用、反复规划。如果每一步都调用超大 reasoning model，成本会非常高。蒸馏小模型可以承担一部分：

- 简单推理；
- 任务拆解；
- 格式化输出；
- 初步代码分析；
- 内部流程判断；
- 低风险自动化步骤。

超大模型则只在关键复杂节点介入。

这篇论文最值得记住的 5 个结论

1. **Reasoning 可以通过 RL 被激发，而不一定完全依赖人工 CoT SFT。**
只要任务可验证、奖励可靠、base model 足够强，模型可以自发学出反思、验证、换方法等推理行为。
2. **R1-Zero 是“纯 RL 激发 reasoning”的关键实验。**
它证明了不用先做人类推理示范，模型也能通过 outcome reward 学会更长、更复杂的推理策略。
3. **aha moment 是模型自我反思能力出现的行为标志。**
它不是神秘意识，而是模型在训练中学会“等等，我可能错了，我要重新检查”。

4. GRPO 是让大规模 reasoning RL 更可行的工程方法。

它用组内相对奖励替代 PPO 里的 value model，降低算力和显存负担，也更适合最终答案奖励。

5. 蒸馏小模型让 reasoning 能力真正产业化。

它把超大模型的推理能力迁移到更便宜、更易部署的小模型中，使私有化、本地化、垂直行业应用成为可能。

最通俗的类比

DeepSeek-R1-Zero 像一个没有看过标准答案详解的学生，但他有大量题可以刷，而且每道题都知道最终对错。

一开始，他可能乱写；后来他发现：

- 多想一会儿更容易对；
- 中途检查更容易对；
- 发现不对就重来更容易对；
- 换一种方法更容易对；
- 最后把答案格式写清楚更容易得分。

于是，他自然发展出一套“推理习惯”。

DeepSeek-R1 则是在这个学生已经很会解难题之后，又补上了表达训练、语言一致性训练、通用问答训练和安全偏好训练，让他不只是会刷奥数题，也更像一个能和人正常交流的助手。

蒸馏小模型则是把这个“解题高手”的解题过程整理成教材，教给一批更小、更便宜、更容易部署的学生。这样 reasoning 能力就不再只属于巨型模型，而可以进入真实业务系统。